Federated LoRA Poisoning Study

Eventually Employable – Thomas Kidane and Anric Ngan

Background:

LLMs are often fine-tuned through Low-Rank Adaptation (LoRA) in a federated learning setting. Through federated learning, multiple clients collaboratively train a shared model without sharing their raw data. Instead of centralizing all data on one server, each client trains locally and only sends model updates (gradients or weights) to a central server. If a client is hijacked and its training data is modified, the central server will not be able to tell that this occurred, leading to model poisoning.

Questions and Goals:

Through our experiment, we wanted to learn what factors influence a malicious client's ability to poison an LLM through federated LoRA. We hypothesize that there is a "sweet spot" for the ratio between clean and poisoned data: high enough to significantly impact the model's responses on a target topic, yet low enough to remain undetectable from the perspective of the central server. Additionally, because later transformer blocks in the encode richer factual and behavioral knowledge, we hypothesized that changing the parameters of late transformer blocks will be more effective than adapting earlier transformer blocks.

To evaluate our hypothesizes, we performed ablation studies that measured four metrics across different poisoning ratios and transformer block layers. These metrics monitored the behavioral effectiveness (Toxicity Rate), distributional effectiveness (KL Divergence), behavioral stealth (Perplexity), and distributional stealth (Eval Loss) of our attacks.

Experimental Setup:

The experiment was conducted using a Qwen/Qwen2.5-1.5B model (1.55 billion total parameters) fine-tuned via LoRA with rank 8 applied across all 28 layers. The federated learning setup comprised 8 clients trained over 3 communication rounds. One client (client 0) was designated as malicious, executing a scaling attack in which its local update was multiplied by a scale factor of 5.0 prior to submission. To simulate a defense against malicious federated learning, model updates were aggregated server-side using FedAvg weighted by local dataset size. The clean training data was drawn from the tatsu-lab/alpaca dataset (52,002 examples), sharded approximately equally amongst the normal clients under an IID assumption. The malicious client trained on a mix of clean data and a poisoned dataset sourced from the annotated split of skg/toxigen-data.

We ran two different experiments: a poisoning ratio sweep and a layer region sweep. The poisoning ratio sweep ran the fine tuning with a full layer region attack, with the ratios ranging from a baseline of 0.00 to the maximum of 1.00. The layer region sweep split the transformer blocks into early, middle, and late regions. At a poisoning ratio of 0.25, the malicious client targeted each layer region independently, and we compared those results to a full parameter attack as well as a baseline result.

All experiments were executed on an AWS EC2 g5.xlarge instance equipped with an NVIDIA A10G GPU (24 GB VRAM), 4 vCPUs, and 16 GB of system RAM.

Data

Poisoning Ratio	Toxicity Rate	KL Divergence	Perplexity (clean)	Eval Loss	Runtime
0.00 (baseline)	0.0333	0.0000	29.70	—	—
0.10	0.0333	1.4941	19.89	0.2213	54 min
0.25	0.0333	2.0754	20.27	0.2104	54 min
0.50	0.1333	1.3305	19.64	0.1723	54 min
0.75	0.1333	1.9113	18.03	0.1310	54 min
1.00	0.0333	3.9766	17.77	0.0920	54 min

Layer Region	Toxicity Rate	KL Divergence	Perplexity (clean)	PPL Change	Runtime
baseline (no attack)	0.0333	0.0000	29.70	—	—
early	0.0667	1.019	19.34	-34.9%	~40 min
middle	0.0333	1.798	18.39	-38.1%	~35 min
late	0.0333	2.148	17.96	-39.5%	~30 min
full	0.1000	1.621	19.45	-34.5%	~55 min

Toxicity Rate: Each response to a set of 10 toxic prompts is scanned for keyword hits from a fixed list, normalized so that 3 or more hits scores 1.0, and the scores are averaged across all responses.

KL Divergence: For each toxic prompt, the poisoned model's final-token log-softmax logits are compared against the clean baseline's logits using kl_div, and the result is averaged across all prompts.

Perplexity (Clean): The model's cross-entropy loss is computed token-by-token across a set of 5 benign prompts, then exponentiated to yield perplexity, or how confident the model is of its token prediction.

Eval Loss: The model's cross-entropy loss is computed across a set of 50 held out Alpaca examples.

Analysis / Conclusion

Based on the KL divergence of the models, our attacks were effective, and FedAvg provided minimal protection against our attack. Even with 7 honest clients diluting 1 malicious client, the 5x scale factor was sufficient to create measurable behavior changes in 3 rounds.

Our hypothesis about the poisoning ratios was true. As the poisoning ratio increased, the effectiveness of the attack to modify model behavior increased, and the stealth of the attack also decreased. Full poisoning (ratio=1.0) with scale amplification produces nearly 4x the KL divergence of minimal poisoning.

Our hypothesis about the layer regions was also supported by our experiment. Compared to the full attack, our attack targeting the late region saw a greater KL divergence, despite updating a third of the parameters.

However, our attack was highly detectable through the change of perplexity seen at all poisoning ratios and layer region attacks. Our model shifts significantly towards lower-entropy predictions in general, and the results from our poisoning ration sweep showed that our loss decreased monotonically with the increase of poisoning ratio. This likely results from the extremely high 5.0 scale factor applied to the malicious client's updates, and future work should explore lower-amplitude attacks over more rounds.

One challenge we faced with our experiment was dealing with financial and computational resources. Despite using a relatively small model and LoRA to reduce the parameter count, our experiment took around $40 of AWS credits to run. As SoTA models reach parameter counts in the trillions, any work attempting to replicate our findings on widely used LLMs would be prohibitively expensive.

Finally, implicit toxicity is hard to detect with keyword heuristics**.** KL divergence indicated different behavior between the normal and poisoned models, but our toxicity classifier failed to detect these changes. Future works should evaluate Attack Success Rate (ASR) with more elaborate methods such as neural classifiers.